# Q4. Data Transformation

Original Aim: To transform the Iris dataset into a form suitable for analysis or machine learning. Convert the Species column into numerical indicator columns using one-hot encoding. Standardize the four numerical attributes and display the transformed data to observe how the values have changed.

## Aim

One-hot encode Species, standardize all four measurements and display the transformed data.

## Requirements

Python, Pandas and scikit-learn StandardScaler. References: [D4], [D5].

## Essential theory

One-hot encoding creates one indicator column per category: 1 means that row belongs to that species; 0 means it does not. It avoids imposing a false numeric ordering on labels. Standardization transforms a measurement using z=(x-mean)/scale, where scale is its population-style standard deviation (ddof=0) for StandardScaler. It centres the feature near zero and gives it unit standard deviation; it does not make every distribution normal or bound values to [0,1].

## Step-by-step procedure

1. Build the original Iris DataFrame.
2. Encode Species separately with get_dummies and integer indicator values.
3. Fit StandardScaler on the four measurement columns for this descriptive transformation exercise.
4. Convert the transformed array back to a DataFrame with the original feature names and matching index.
5. Concatenate standardized features with the indicators. Display both parts, the combined shape and numerical checks.

## Complete Python program

```
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
features = [
    "SepalLengthCm", "SepalWidthCm",
    "PetalLengthCm", "PetalWidthCm"
]
species = ["Iris-" + name for name in iris.target_names]
df = pd.DataFrame(iris.data, columns=features)
df["Species"] = pd.Categorical.from_codes(
    iris.target, categories=species
)

# Encoding and numerical scaling are separate operations.
encoded = pd.get_dummies(df["Species"], prefix="Species", dtype=int)
scaler = StandardScaler()
scaled = pd.DataFrame(
    scaler.fit_transform(df[features]),
    columns=features, index=df.index
)
transformed = pd.concat([scaled, encoded], axis=1)
print("INDICATOR COLUMNS: first five rows")
print(encoded.head().to_string(index=False))
print("STANDARDIZED FEATURES: first five rows")
print(scaled.head().round(4).to_string(index=False))
print("Transformed shape:", transformed.shape)
print("Transformed columns:", transformed.columns.tolist())
print("Scaled means:", scaled.mean().round(6).tolist())
print("Scaled std, ddof=0:", scaled.std(ddof=0).round(6).tolist())
```

## Code explanation

- `pd.get_dummies(..., dtype=int)` gives explicit 0/1 columns; it retains all three species indicators here.
- `scaler.fit_transform` learns each feature's mean/scale, then applies them. Q4 describes the whole dataset, so a full-data fit is appropriate here.
- `pd.concat(..., axis=1)` joins columns by row index. Preserving df.index avoids accidental row misalignment.
- `scaled.std(ddof=0)` matches StandardScaler's convention; ddof=1 would give a slightly different value.
- The Species indicators are included to demonstrate the assigned transformation. Do not feed these target-derived columns back as predictors in Q11/Q13: that would reveal the answer to the classifier. Those tasks use only the four numerical features.

## Verified output

```
INDICATOR COLUMNS: first five rows
 Species_Iris-setosa  Species_Iris-versicolor  Species_Iris-virginica
                   1                        0                       0
                   1                        0                       0
                   1                        0                       0
                   1                        0                       0
                   1                        0                       0
STANDARDIZED FEATURES: first five rows
 SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
       -0.9007        1.0190        -1.3402       -1.3154
       -1.1430       -0.1320        -1.3402       -1.3154
       -1.3854        0.3284        -1.3971       -1.3154
       -1.5065        0.0982        -1.2834       -1.3154
       -1.0218        1.2492        -1.3402       -1.3154
Transformed shape: (150, 7)
Transformed columns: ['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm',
  'Species_Iris-setosa', 'Species_Iris-versicolor', 'Species_Iris-virginica']
Scaled means: [-0.0, -0.0, -0.0, -0.0]
Scaled std, ddof=0: [1.0, 1.0, 1.0, 1.0]
```

The seven columns are the four standardized measurements plus Species_Iris-setosa, Species_Iris-versicolor and Species_Iris-virginica. The first five rows all belong to setosa, so their indicator values are identical. All three indicator columns vary over the complete dataset. A printed -0.0 is floating-point rounding of a value effectively equal to zero.

## Observation

For the first row, sepal length is below the dataset mean, so its standardized value is negative; sepal width is above its mean, so its value is positive. All four scaled means are approximately zero and population standard deviations are one. Species is represented by separate category indicators rather than an artificial ranked code.

## Result

Encoding and scaling were completed as distinct operations, producing a 150-by-7 descriptive table. In a predictive evaluation, split first and fit preprocessing on training rows only; see Q11/Q13.

## Common errors and troubleshooting

- Could not convert string to float: pass only df[features] to StandardScaler, not Species.
- All dummy columns appear True/False: this is a valid default in some Pandas versions; dtype=int requests 0/1 output.
- Scaled standard deviation looks slightly above one: confirm whether you computed sample ddof=1 rather than population ddof=0.

## Viva questions and answers

- Does one-hot encoding rank the species? No; it represents membership independently.
- Why can standardized values be negative? They correspond to values below the fitted mean.
- Does StandardScaler guarantee a normal distribution? No; it changes centre and scale, not distributional shape.
- Why exclude species indicators from a species classifier's inputs? They directly encode its target and create leakage.